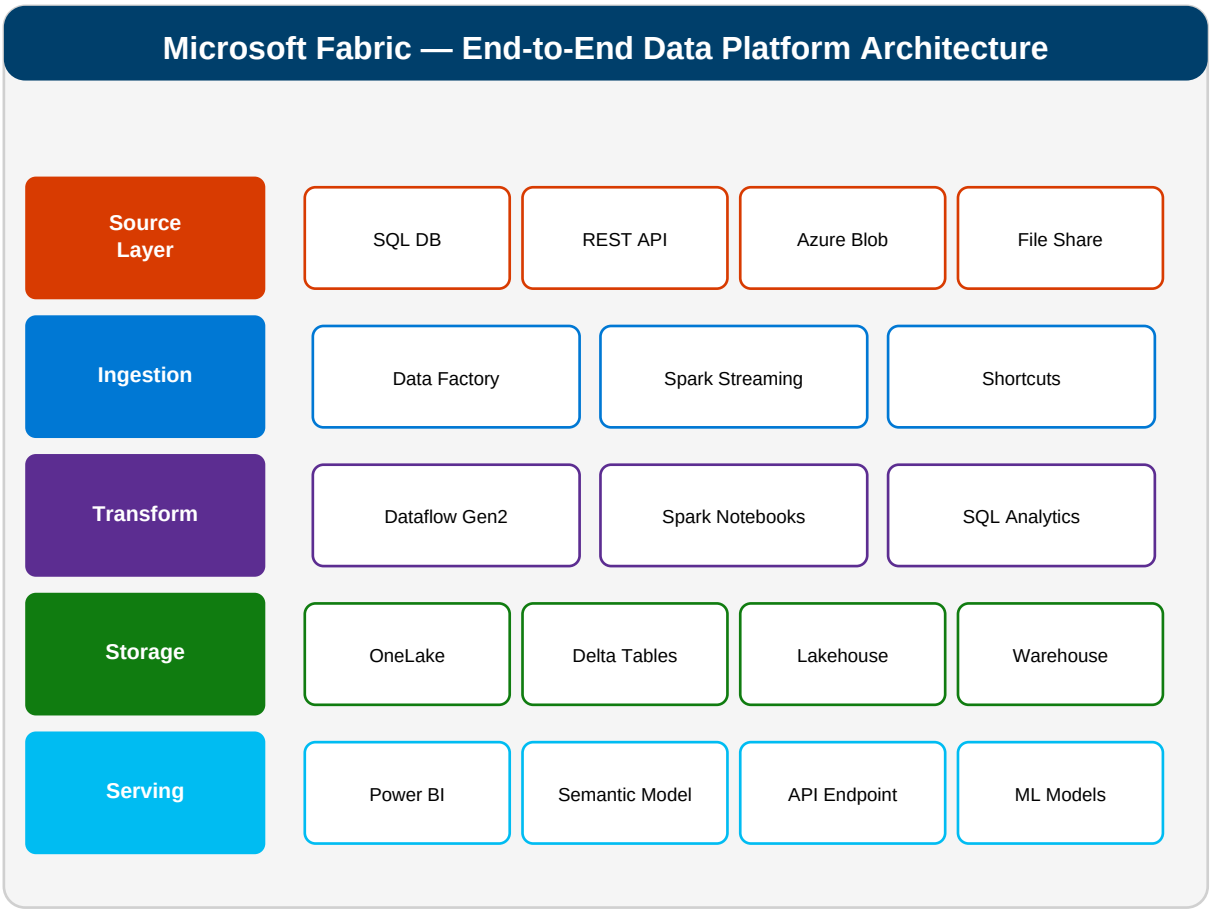
Microsoft Fabric

ETL, Pipelines & Data Engineering

Complete Practical Guide with Scenarios, Code & Architecture



Edition: 2024 | **Platform:** Microsoft Fabric GA

Covering: Data Factory • Lakehouse • Dataflows Gen2 • Spark • Pipeline Orchestration

Table of Contents

Chapter 1: Microsoft Fabric Overview & Architecture

1.1 What is Microsoft Fabric?

Microsoft Fabric is a unified analytics platform that combines data engineering, data integration, data warehousing, real-time analytics, and business intelligence into a single SaaS offering. Built on top of Azure, Fabric provides a **OneLake** storage foundation with an open Delta-Parquet format, enabling zero-copy data sharing across all workloads without duplication.

At its core, Fabric eliminates the traditional need to stitch together multiple Azure services (ADF, Synapse, ADLS, Power BI) by providing a single, integrated experience within one tenant. Every workload shares the same compute, storage, security model, and monitoring surface.

1.2 Core Components & Services

Service	Purpose
Data Factory	Pipelines, Dataflows Gen2, data movement & orchestration
Lakehouse	Delta Lake storage + Spark compute + SQL analytics endpoint
Data Warehouse	T-SQL serverless warehouse on OneLake Delta tables
Synapse Analytics	Apache Spark pool for large-scale data transformation
Real-Time Intel.	EventStream, KQL Database, Real-Time Dashboards
Power BI	Semantic models, reports, paginated reports
Data Activator	Automated triggers & actions based on data conditions
Data Science	ML experiments, models, AutoML via MLflow integration

1.3 OneLake — The Unified Storage Layer

OneLake is the single logical data lake underpinning all Fabric workloads. It is hierarchically organized as **Tenant** → **Workspace** → **Item (Lakehouse/Warehouse/etc.)**. All data is stored in **Delta-Parquet** format, accessible via ABFS, REST, or Shortcuts.

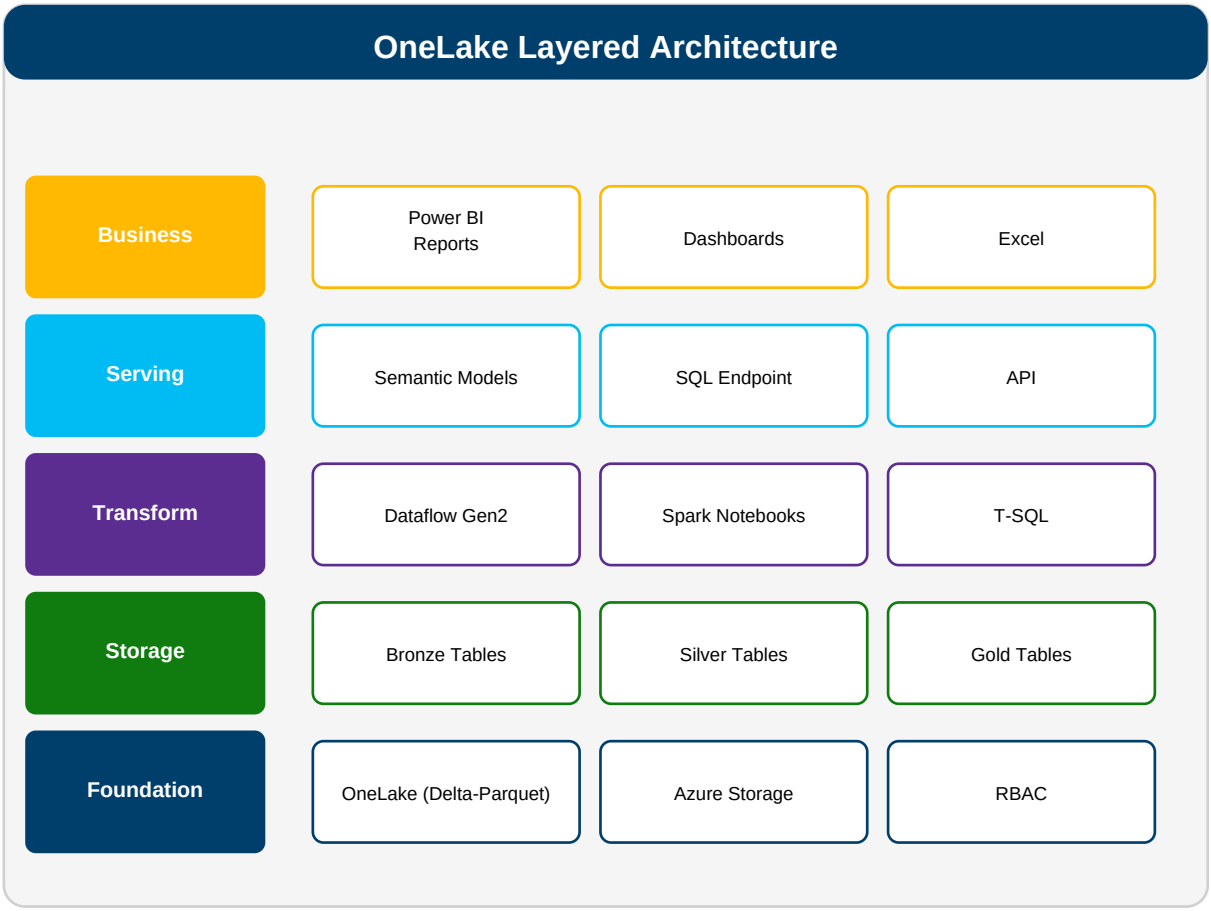


Figure 1.1 OneLake acts as the single source of truth across all Fabric workloads.

1.4 Fabric Capacity Units (CU) & Licensing

SKU	Capacity	Use Case
F2	2 CU	Dev/test, small Dataflows, light notebook work
F4	4 CU	Small PoC, basic pipelines, limited concurrent runs
F8	8 CU	Small production workloads, moderate Spark jobs
F16	16 CU	Standard production, multiple concurrent pipelines
F32	32 CU	Large ETL, moderate Spark streaming, heavy DWH queries
F64	64 CU	Enterprise production, large-scale Spark + Power BI Premium
F128	128 CU	Very large scale, global rollouts, multiple workspaces

Chapter 2: Data Factory & Pipeline Fundamentals

2.1 Pipeline Concepts & Terminology

A **Pipeline** in Microsoft Fabric Data Factory is a logical container for a workflow of activities. Each pipeline can be triggered manually, on a schedule, via event triggers, or invoked from a parent pipeline. Pipelines are defined in JSON (ARM-compatible) and can be version-controlled in Git.

- **Activity:** A single unit of work — Copy Data, Notebook, Stored Procedure, etc.
- **Linked Service:** Connection definition to a data source or compute resource
- **Dataset:** Named reference to data within a linked service (schema + location)
- **Integration Runtime:** Compute engine: Azure IR, Self-Hosted IR, or SSIS IR
- **Trigger:** Mechanism to start a pipeline: Schedule, Tumbling Window, Event
- **Parameter:** Pipeline-level input value passed at runtime
- **Variable:** Pipeline-scoped mutable value set by Set Variable activity
- **Run ID:** Unique GUID for each pipeline execution instance

2.2 Activity Types Reference

Activity	Description
Copy Data	Move data between source and sink connectors
Notebook	Execute a Spark Notebook with optional parameters
Dataflow	Run a Dataflow Gen2 transformation
Stored Procedure	Execute a SQL stored procedure on a warehouse/database
Lookup	Read a single row/value to use in downstream activities
Get Metadata	Retrieve file/folder properties (size, last modified, etc.)
ForEach	Iterate over an array with parallel/sequential execution
If Condition	Branch pipeline based on boolean expression
Switch	Multi-branch execution based on expression value
Until	Loop activities until a condition becomes true
Wait	Pause execution for N seconds
Set Variable	Assign a value to a pipeline variable
Append Variable	Add element to an array variable

Execute Pipeline	Invoke a child pipeline (sync or async)
Fail	Explicitly fail the pipeline with a custom message
Web Activity	Call REST API endpoints from within the pipeline
Script	Execute T-SQL scripts on a Warehouse endpoint
Delete	Delete files or folders in storage

2.3 Pipeline Run Dependencies (Success / Failure / Completion)

Each activity in a pipeline can have dependency conditions on upstream activities. The four conditions are **Success**, **Failure**, **Skipped**, and **Completion**. These enable sophisticated error-handling patterns such as alerting on failure while still marking the pipeline as succeeded, or running cleanup activities regardless of outcome.

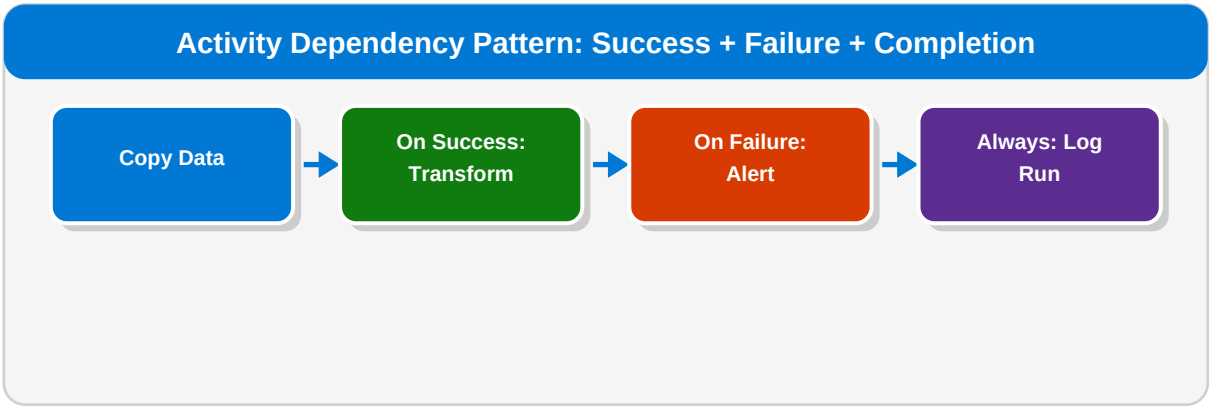


Figure 2.1 Parallel success/failure paths with a final completion step.

2.4 Pipeline Parameters & Variables

Parameters are immutable inputs passed at trigger or invocation time. Variables are mutable and can be updated during runtime using the *Set Variable* activity. Best practice is to use parameters for anything that changes between environments (e.g., environment name, date ranges) and variables for loop counters or accumulators.

Pipeline JSON — Parameter & Variable Definition

```
{
  "parameters": {
    "SourceSchema":    { "type": "string", "defaultValue": "dbo" },
    "WatermarkDate":   { "type": "string", "defaultValue": "1900-01-01" },
    "BatchSize":       { "type": "int",    "defaultValue": 10000 }
  },
  "variables": {
    "RowsProcessed":   { "type": "integer", "defaultValue": 0 },
    "HasMorePages":    { "type": "boolean", "defaultValue": true },
    "CurrentPage":     { "type": "integer", "defaultValue": 1 }
  }
}
```

Dynamic Content Expression — Using Parameters

```
-- Reference parameter in Copy Activity Source query
@concat(
  'SELECT * FROM ', pipeline().parameters.SourceSchema,
  '.Orders WHERE ModifiedDate > ''',
  pipeline().parameters.WatermarkDate, ''''
)

-- Reference system variable
@pipeline().RunId
@pipeline().TriggerTime
@utcNow()
@formatDateTime(utcNow(), 'yyyy-MM-dd')
```

Chapter 3: Scenario 1 — SQL Server to Lakehouse ETL

SCENARIO 1 | SQL Server to Fabric Lakehouse — Incremental ETL

Business Problem: A retail company has a 50-table on-premises SQL Server OLTP database. They need to ingest it daily into Microsoft Fabric Lakehouse using an incremental watermark strategy, then transform it for downstream Power BI reporting.

Tables: Orders, OrderLines, Customers, Products, Inventory (50+ tables)

Volume: ~2M new/updated rows per day across all tables

SLA: Data must be available in Gold layer by 06:00 UTC daily

3.1 Architecture Design

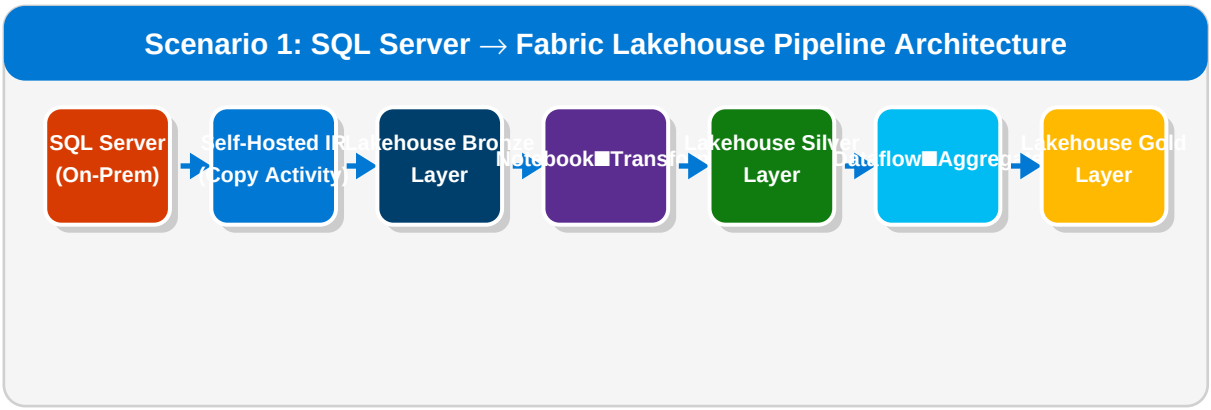


Figure 3.1 End-to-end data flow from on-premises SQL Server to Gold layer.

3.2 Metadata-Driven Watermark Table

Rather than hard-coding table names in each pipeline, we use a control table in the Fabric Warehouse to store watermark values and table configurations. This makes the pipeline fully dynamic and requires zero code changes when adding new source tables.

SQL — Create Watermark Control Table

```
-- Run in Fabric Warehouse SQL Endpoint
CREATE TABLE dbo.etl_watermark (
    table_id        INT IDENTITY(1,1) PRIMARY KEY,
    source_schema   VARCHAR(50)  NOT NULL,
    source_table    VARCHAR(100) NOT NULL,
    watermark_col   VARCHAR(100) NOT NULL DEFAULT 'ModifiedDate',
    last_watermark  DATETIME2    NOT NULL DEFAULT '1900-01-01',
```

```
    target_table    VARCHAR(100) NOT NULL,
    is_active       BIT          NOT NULL DEFAULT 1,
    load_mode       VARCHAR(20)  NOT NULL DEFAULT 'INCREMENTAL', -- FULL | INCREMENTAL
    batch_size      INT          NOT NULL DEFAULT 50000
);

-- Seed with tables to ingest
INSERT INTO dbo.etl_watermark
  (source_schema, source_table, watermark_col, target_table, load_mode)
VALUES
  ('dbo','Orders',    'ModifiedDate', 'bronze_orders',    'INCREMENTAL'),
  ('dbo','Customers', 'UpdatedAt',    'bronze_customers', 'INCREMENTAL'),
  ('dbo','Products',  'ModifiedDate', 'bronze_products',  'INCREMENTAL'),
  ('dbo','Inventory', 'LastUpdated',  'bronze_inventory', 'FULL');
```

3.3 Master Pipeline — ForEach with Lookup

The master pipeline reads the watermark control table, then iterates over each active table using a **ForEach** activity set to *parallel execution* with a batch count of 4. Inside each iteration, a child pipeline handles the actual copy.

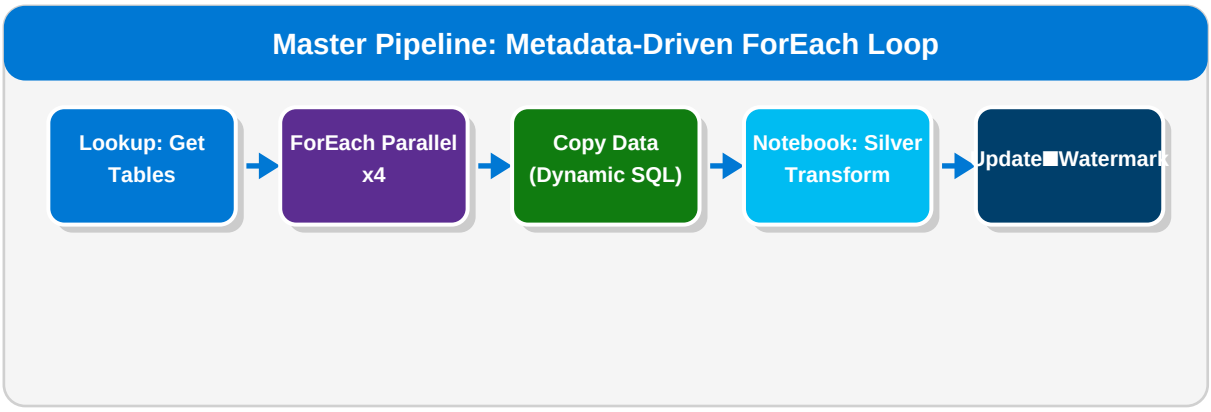


Figure 3.2 Master pipeline with parallel ForEach across all source tables.

Pipeline — Lookup Activity: Get Active Tables

```
// Lookup Activity Settings
{
  'name': 'LookupActiveTableList',
  'type': 'Lookup',
  'settings': {
    'source': {
      'type': 'WarehouseSource',
      'sqlReaderQuery': {
        'value': 'SELECT table_id, source_schema, source_table,
                        watermark_col, last_watermark,
                        target_table, load_mode, batch_size
                  FROM dbo.etl_watermark
                  WHERE is_active = 1
                  ORDER BY table_id',
        'type': 'Expression'
      }
    },
    'firstRowOnly': false
  }
}
```

Pipeline — ForEach Activity Dynamic Copy Source SQL

```
// Dynamic query expression used inside ForEach
// item() refers to current row from Lookup
@if(
  equals(item().load_mode, 'FULL'),
  concat('SELECT * FROM ', item().source_schema, '.', item().source_table),
  concat(
    'SELECT * FROM ', item().source_schema, '.', item().source_table,
    ' WHERE ', item().watermark_col,
    ' > ''', item().last_watermark, '''',
    ' AND ',  item().watermark_col,
    ' <= ''', formatDateTime(utcNow(), 'yyyy-MM-dd HH:mm:ss'), ''''
  )
)
```

3.4 Silver Transformation — PySpark Notebook

PySpark — Bronze to Silver: Orders Table

```
# Fabric Notebook: bronze_to_silver_orders.ipynb
# Parameters cell (tagged 'parameters' for pipeline injection)
target_table = 'bronze_orders'
watermark_date = '1900-01-01'

from pyspark.sql import functions as F
from pyspark.sql.types import *
from delta.tables import DeltaTable

LAKEHOUSE = 'abfss://workspace@onelake.dfs.fabric.microsoft.com/lakehouse.Lakehouse'

# --- Read Bronze ---
df_bronze = spark.read.format('delta') \
    .load(f'{LAKEHOUSE}/Tables/{target_table}')

# --- Cleanse & Enrich ---
df_silver = (df_bronze
    .filter(F.col('OrderID').isNotNull())
    .filter(F.col('OrderDate') >= '2000-01-01')
    .withColumn('OrderDate',    F.to_date('OrderDate'))
    .withColumn('ShipDate',     F.to_date('ShipDate'))
    .withColumn('TotalAmount',  F.round(F.col('TotalAmount'), 2))
    .withColumn('DaysToShip',
        F.datediff(F.col('ShipDate'), F.col('OrderDate')))
    .withColumn('IsLateShipment',
        F.when(F.col('DaysToShip') > 7, True).otherwise(False))
    .withColumn('_ingestion_ts', F.current_timestamp())
    .withColumn('_source',       F.lit('sql_server_oltp'))
    .dropDuplicates(['OrderID'])
)

# --- Upsert (MERGE) into Silver Delta Table ---
silver_path = f'{LAKEHOUSE}/Tables/silver_orders'

if DeltaTable.isDeltaTable(spark, silver_path):
    dt = DeltaTable.forPath(spark, silver_path)
    dt.alias('target').merge(
        df_silver.alias('source'),
        'target.OrderID = source.OrderID'
    ).whenMatchedUpdateAll(
    ).whenNotMatchedInsertAll(
    ).execute()
else:
```

```
    df_silver.write.format('delta') \
        .mode('overwrite') \
        .option('overwriteSchema', 'true') \
        .save(silver_path)

print(f'Silver orders updated. Rows processed: {df_silver.count()}')
```

3.5 Update Watermark After Successful Load

Pipeline — Script Activity: Update Watermark

```
-- Script Activity (T-SQL) — runs after successful Notebook
UPDATE dbo.etl_watermark
SET    last_watermark = GETUTCDATE()
WHERE  source_table   = '@{item().source_table}'
AND    source_schema  = '@{item().source_schema}';

-- Log the pipeline run
INSERT INTO dbo.etl_run_log
  (run_id, table_name, start_time, end_time, rows_copied, status)
VALUES
  ('@{pipeline().RunId}',
   '@{item().source_table}',
   '@{activity(''CopyData'').output.executionDetails[0].start}',
   GETUTCDATE(),
   @{activity(''CopyData'').output.rowsCopied},
   'SUCCESS');
```

Chapter 4: Scenario 2 — REST API Ingestion Pipeline

SCENARIO 2 | REST API Ingestion with OAuth2 & Pagination

Business Problem: Ingest data from a third-party e-commerce REST API that requires OAuth2 authentication, returns paginated JSON responses (100 records/page), and needs to be landed in the Lakehouse as Delta tables daily.

API: Shopify-like REST API | **Auth:** OAuth2 Client Credentials

Endpoints: /orders, /customers, /products, /inventory

Volume: Up to 50,000 records per endpoint per day

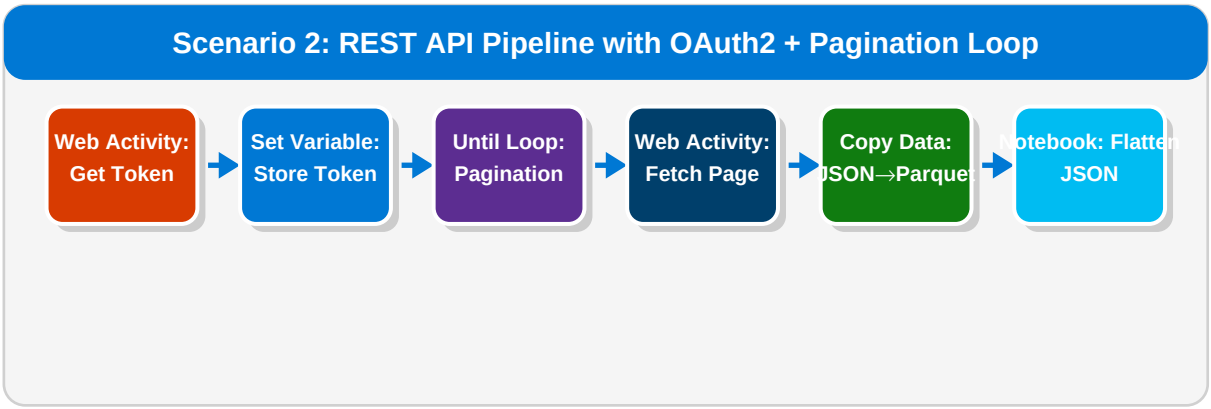


Figure 4.1 REST API ingestion pipeline with token refresh and pagination handling.

4.1 OAuth2 Token Acquisition (Web Activity)

Pipeline — Web Activity: Get OAuth2 Token

```
// Web Activity: GetOAuthToken
{
  'name': 'GetOAuthToken',
  'type': 'WebActivity',
  'settings': {
    'url': 'https://api.example.com/oauth/token',
    'method': 'POST',
    'headers': {
      'Content-Type': 'application/x-www-form-urlencoded'
    },
    'body': {
      'value': "grant_type=client_credentials&client_id=@{linkedService().clientId}&client_sec
ret=@{linkedService().clientSecret}&scope=read",
      'type': 'Expression'
    }
  }
}

// Set Variable — store token from Web Activity output
```

```
// Expression:
@activity('GetOAuthToken').output.access_token
```

4.2 Pagination Loop with Until Activity

Pipeline — Until Activity: Paginate API

```
// Until Activity: PaginateUntilEmpty
// Exit condition — no more pages:
@equals(variables('HasMorePages'), false)

// Inside Until — Web Activity: FetchPage
URL Expression:
@concat(
  'https://api.example.com/v1/orders',
  '?page=', variables('CurrentPage'),
  '&per_page=100',
  '&updated_at_min=', pipeline().parameters.WatermarkDate
)

// Headers Expression:
@concat('Bearer ', variables('AccessToken'))

// After FetchPage — Set Variable: IncrementPage
@string(add(int(variables('CurrentPage')), 1))

// Set Variable: CheckHasMorePages
// If response returns empty array, set HasMorePages = false
@if(
  equals(length(activity('FetchPage').output.orders), 0),
  false,
  true
)
```

4.3 PySpark — Flatten Nested JSON

PySpark — Flatten & Store REST API Response

```
# Notebook: flatten_api_orders.ipynb
from pyspark.sql import functions as F
from pyspark.sql.types import *

LANDING = 'abfss://workspace@onelake.dfs.fabric.microsoft.com/lakehouse.Lakehouse'

# Read raw JSON files landed by Copy Activity
df_raw = spark.read.option('multiline', 'true') \
              .json(f'{LANDING}/Files/api_landing/orders/*.json')

# Explode orders array
df_orders = df_raw.select(F.explode('orders').alias('order'))

# Flatten nested structure
df_flat = df_orders.select(
    F.col('order.id').alias('order_id'),
    F.col('order.created_at').alias('created_at'),
    F.col('order.updated_at').alias('updated_at'),
    F.col('order.email').alias('customer_email'),
    F.col('order.total_price').cast('double').alias('total_price'),
    F.col('order.currency').alias('currency'),
    F.col('order.financial_status').alias('payment_status'),
```

```
    F.col('order.fulfillment_status').alias('fulfillment_status'),
    # Nested: shipping address
    F.col('order.shipping_address.city').alias('ship_city'),
    F.col('order.shipping_address.country_code').alias('ship_country'),
    # Nested: line_items array — explode separately
    F.col('order.line_items').alias('line_items'),
    F.current_timestamp().alias('_ingested_at')
)

# Explode line items to separate table
df_lines = df_orders.select(
    F.col('order.id').alias('order_id'),
    F.explode('order.line_items').alias('line')
).select(
    'order_id',
    F.col('line.id').alias('line_id'),
    F.col('line.product_id').alias('product_id'),
    F.col('line.quantity').cast('int').alias('quantity'),
    F.col('line.price').cast('double').alias('unit_price'),
)

# Write as Delta
df_flat.drop('line_items').write.format('delta') \
       .mode('overwrite').save(f'{LANDING}/Tables/bronze_api_orders')

df_lines.write.format('delta') \
        .mode('overwrite').save(f'{LANDING}/Tables/bronze_api_order_lines')

print(f'Orders: {df_flat.count()}, Lines: {df_lines.count()}')
```

4.4 Error Handling & Retry Pattern

REST APIs often return transient errors (429 Too Many Requests, 503 Service Unavailable). Fabric pipelines support **retry policies** per activity and **on-failure branching** to handle these gracefully.

Pipeline JSON — Activity Retry & Timeout Policy

```
{
  'name': 'FetchAPIPage',
  'type': 'WebActivity',
  'policy': {
    'timeout': '00:02:00',      // 2-minute timeout per call
    'retry': 3,                  // Retry up to 3 times
    'retryIntervalInSeconds': 30, // 30s between retries
    'secureOutput': false
  },
  'onInactive': 'Continue'
}

// On-Failure branch — send alert email
// Connect Fail activity with dependency: ['FetchAPIPage', 'Failed']
{
  'name': 'SendAlertEmail',
  'type': 'WebActivity',
  'dependsOn': [{ 'activity': 'FetchAPIPage', 'dependencyConditions': ['Failed'] }],
  'settings': {
    'url': 'https://prod.logic.azure.com/workflows/...',
    'method': 'POST',
    'body': {
      'subject': 'Pipeline Failed',
```

```
      'message': '@{activity(''FetchAPIPage'').error.message}'
    }
  }
}
```

Chapter 5: Scenario 3 — Dataflows Gen2 Transformation

SCENARIO 3 | Customer 360 — Multi-Source Dataflow Gen2

Business Problem: The analytics team needs to build a Customer 360 view by joining data from four sources: CRM (SQL), marketing platform (API), support tickets (CSV in SharePoint), and web clickstream (Lakehouse Delta table). Result must feed a Power BI Semantic Model.

Sources: SQL CRM + Marketing API + SharePoint CSV + Lakehouse Delta

Output: Gold layer Lakehouse table: gold_customer_360

SCD: Type 2 history tracking on customer segment changes

5.1 Dataflow Gen2 Architecture

Dataflows Gen2 in Microsoft Fabric are built using Power Query (M language) under the hood, but now execute on a Spark-based engine for large-scale data, called **Staging**. When staging is enabled, each query step is pushed down to Spark, enabling billion-row transformations that weren't possible in classic Dataflows.

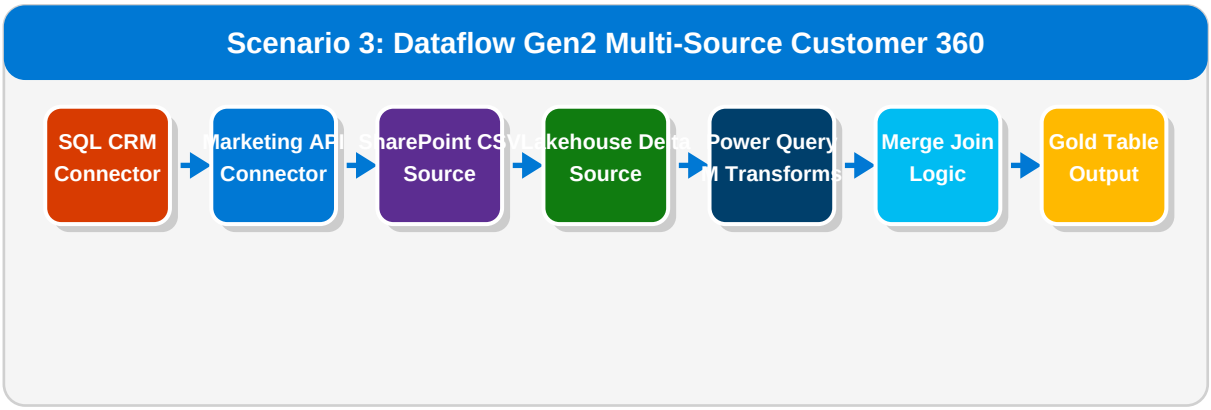


Figure 5.1 Dataflow Gen2 combining four source systems into a unified customer view.

5.2 Power Query M — Core Transformation Steps

Power Query M — CRM Customer Query

```
// Query: CRM_Customers
let
    Source = Sql.Database('sqlserver.database.windows.net', 'CRM_DB'),
    dbo_Customers = Source{[Schema='dbo',Item='Customers']}[Data],

    // Filter active customers only
    FilterActive = Table.SelectRows(dbo_Customers,
        each [Status] = 'Active' and [IsDeleted] = false),
```

```
    // Rename and select columns
    Renamed = Table.RenameColumns(FilterActive, {
        {'CustomerID',   'crm_customer_id'},
        {'EmailAddress', 'email'},
        {'FirstName',    'first_name'},
        {'LastName',     'last_name'},
        {'Segment',      'crm_segment'},
        {'AccountValue', 'account_value'}
    }),

    // Derive full name
    AddFullName = Table.AddColumn(Renamed, 'full_name',
        each [first_name] & ' ' & [last_name], type text),

    // Normalize email
    NormalizeEmail = Table.TransformColumns(AddFullName,
        {{'email', Text.Lower, type text}})
in
    NormalizeEmail
```

Power Query M — Merge Four Sources into Customer 360

```
// Query: Customer_360_Merged
let
    // Reference individual source queries
    CRM        = CRM_Customers,
    Marketing  = Marketing_Profiles,
    Support    = Support_Tickets_Aggregated,
    Clickstream = Clickstream_Summary,

    // Join CRM + Marketing on email
    JoinMarketing = Table.NestedJoin(
        CRM, {'email'},
        Marketing, {'email_normalized'},
        'MarketingData', JoinKind.LeftOuter
    ),
    ExpandMktg = Table.ExpandTableColumn(JoinMarketing,
        'MarketingData',
        {'campaign_segment', 'email_opens_30d', 'last_campaign_date'}
    ),

    // Join with Support tickets (left outer)
    JoinSupport = Table.NestedJoin(
        ExpandMktg, {'crm_customer_id'},
        Support, {'customer_id'},
        'SupportData', JoinKind.LeftOuter
    ),
    ExpandSupport = Table.ExpandTableColumn(JoinSupport,
        'SupportData',
        {'open_tickets', 'avg_resolution_days', 'csat_score'}
    ),

    // Join with Clickstream (left outer)
    JoinClick = Table.NestedJoin(
        ExpandSupport, {'email'},
        Clickstream, {'user_email'},
        'ClickData', JoinKind.LeftOuter
    ),
    ExpandClick = Table.ExpandTableColumn(JoinClick,
        'ClickData',
```

```
        {'page_views_30d', 'sessions_30d', 'last_visit_date'}
    ),

    // Add composite health score
    AddHealthScore = Table.AddColumn(ExpandClick, 'customer_health_score',
        each
            (if [csat_score] <> null then [csat_score] * 0.4 else 0) +
            (if [page_views_30d] <> null then
                Number.Min([page_views_30d] / 100, 1) * 0.3 else 0) +
            (if [email_opens_30d] <> null then
                Number.Min([email_opens_30d] / 10, 1) * 0.3 else 0)
        , type number),

    // Add ingestion timestamp
    AddTimestamp = Table.AddColumn(AddHealthScore,
        '_refreshed_at', each DateTime.LocalNow(), type datetime)
in
    AddTimestamp
```

5.3 Slowly Changing Dimension Type 2 (SCD Type 2)

For the customer segment attribute, business requires full history tracking (SCD Type 2). When a customer's segment changes (e.g., Bronze → Gold), we close the old record and insert a new one with updated effective dates.

PySpark — SCD Type 2 Implementation for Customer Segment

```
# Notebook: apply_scd2_customer_segment.ipynb
from pyspark.sql import functions as F
from delta.tables import DeltaTable
from pyspark.sql.window import Window

LAKEHOUSE = 'abfss://workspace@onelake.dfs.fabric.microsoft.com/lakehouse.Lakehouse'
SCD2_TABLE = f'{LAKEHOUSE}/Tables/gold_customer_segment_history'

# Read incoming (new) data
df_new = spark.read.format('delta') \
    .load(f'{LAKEHOUSE}/Tables/silver_customers')

# Read existing SCD2 table (current records only)
if DeltaTable.isDeltaTable(spark, SCD2_TABLE):
    df_existing = spark.read.format('delta').load(SCD2_TABLE) \
        .filter(F.col('is_current') == True)

    # Detect changed segments
    df_changed = df_new.alias('new').join(
        df_existing.alias('cur'),
        on='crm_customer_id'
    ).filter(
        F.col('new.crm_segment') != F.col('cur.crm_segment')
    ).select('new.*')

    # Expire old records: set end_date and is_current = false
    dt = DeltaTable.forPath(spark, SCD2_TABLE)
    dt.alias('target').merge(
        df_changed.alias('source'),
        'target.crm_customer_id = source.crm_customer_id AND target.is_current = true'
    ).whenMatchedUpdate(set={
        'end_date':   F.lit(F.current_date()),
        'is_current': F.lit(False)
```

```
    }).execute()

    # Insert new records for changed customers
    df_insert = df_changed \
        .withColumn('start_date', F.current_date()) \
        .withColumn('end_date',   F.lit('9999-12-31').cast('date')) \
        .withColumn('is_current', F.lit(True))

    df_insert.write.format('delta') \
        .mode('append').save(SCD2_TABLE)

    # Also insert brand new customers
    df_existing_ids = df_existing.select('crm_customer_id')
    df_new_customers = df_new.join(df_existing_ids,
        on='crm_customer_id', how='left_anti')
else:
    # First run — insert all as current
    df_new_customers = df_new

df_new_customers \
    .withColumn('start_date', F.current_date()) \
    .withColumn('end_date',   F.lit('9999-12-31').cast('date')) \
    .withColumn('is_current', F.lit(True)) \
    .write.format('delta').mode('append').save(SCD2_TABLE)

print('SCD2 applied successfully.')
```

Chapter 6: Scenario 4 — Spark Notebook ETL Pipeline

SCENARIO 4 | Large-Scale Clickstream Processing — PySpark ETL

Business Problem: Process 500GB+ of daily clickstream event files (JSON) from an Azure Blob container, enrich with product and customer dimension data, compute session metrics, and load into a Gold layer analytics table.

Source: Azure Blob Storage — 10,000+ JSON files/day, ~500GB

Processing: Spark cluster, sessionization, enrichment, aggregation

Output: gold_clickstream_sessions Delta table for Power BI

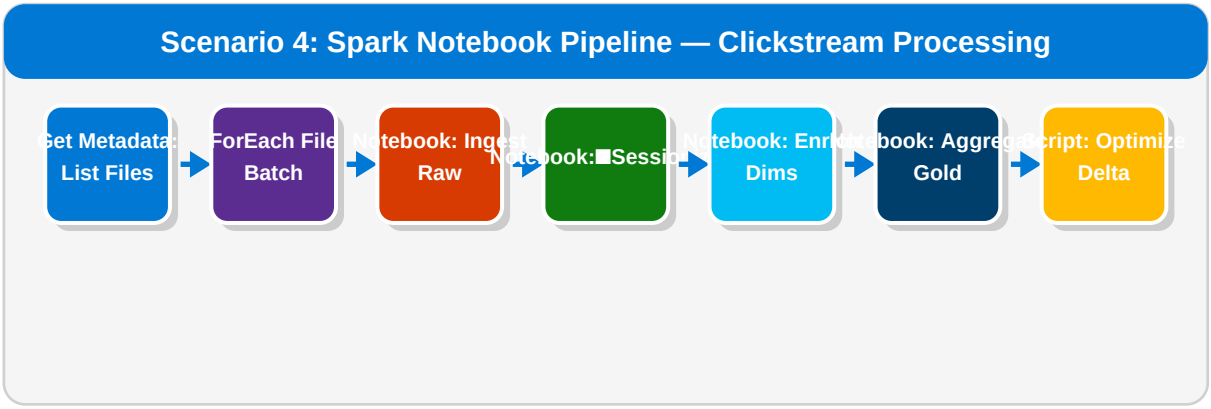


Figure 6.1 Multi-notebook pipeline for large-scale clickstream event processing.

6.1 Ingest Raw Clickstream Events

PySpark — Notebook 1: Raw Ingestion with Schema Enforcement

```
# Notebook: 01_ingest_clickstream.ipynb
from pyspark.sql import functions as F
from pyspark.sql.types import *

# Explicit schema prevents schema inference overhead on 500GB+
EVENT_SCHEMA = StructType([
    StructField('event_id',       StringType(), False),
    StructField('session_id',     StringType(), True),
    StructField('user_id',        StringType(), True),
    StructField('anonymous_id',   StringType(), True),
    StructField('event_type',     StringType(), False),
    StructField('page_url',       StringType(), True),
    StructField('referrer',       StringType(), True),
    StructField('product_id',     StringType(), True),
    StructField('timestamp_utc',  TimestampType(), False),
    StructField('device_type',    StringType(), True),
    StructField('browser',        StringType(), True),
```

```
    StructField('country_code',   StringType(), True),
    StructField('properties',     MapType(StringType(), StringType()), True),
])

BLOB_SOURCE = 'abfss://clickstream@storageaccount.dfs.core.windows.net/raw/'
LANDING     = 'abfss://workspace@onelake.dfs.fabric.microsoft.com/lakehouse.Lakehouse'

# Read with auto-partitioning across Spark workers
df_raw = (spark.read
    .schema(EVENT_SCHEMA)
    .option('badRecordsPath', f'{LANDING}/Files/_bad_records/clickstream')
    .json(f'{BLOB_SOURCE}{processing_date}/')  # processing_date = pipeline param
)

# Partition by event_date for efficient downstream reads
df_partitioned = df_raw \
    .withColumn('event_date', F.to_date('timestamp_utc')) \
    .withColumn('event_hour', F.hour('timestamp_utc'))

# Write to Bronze (append, partition pruning enabled)
(df_partitioned.write
    .format('delta')
    .mode('overwrite')
    .partitionBy('event_date', 'event_hour')
    .option('replaceWhere',
            f"event_date = '{processing_date}'")
    .save(f'{LANDING}/Tables/bronze_clickstream')
)

print(f'Ingested {df_raw.count():,} events for {processing_date}')
```

6.2 Sessionization Logic

PySpark — Notebook 2: Session Attribution using Window Functions

```
# Notebook: 02_sessionize_events.ipynb
from pyspark.sql import functions as F
from pyspark.sql.window import Window

SESSION_TIMEOUT_MINS = 30

df_events = spark.read.format('delta') \
    .load(f'{LANDING}/Tables/bronze_clickstream') \
    .filter(F.col('event_date') == processing_date)

# Define window: user ordered by time
user_time_window = Window \
    .partitionBy('user_id') \
    .orderBy('timestamp_utc')

# Calculate time gap from previous event
df_gaps = df_events.withColumn(
    'prev_event_ts',
    F.lag('timestamp_utc').over(user_time_window)
).withColumn(
    'gap_minutes',
    (F.col('timestamp_utc').cast('long') -
     F.col('prev_event_ts').cast('long')) / 60
)
```

```
# New session starts when gap > 30 min OR first event
df_sessions = df_gaps.withColumn(
    'is_new_session',
    F.when(
        (F.col('prev_event_ts').isNull()) |
        (F.col('gap_minutes') > SESSION_TIMEOUT_MINS),
        1
    ).otherwise(0)
).withColumn(
    'session_seq',
    F.sum('is_new_session').over(user_time_window)
).withColumn(
    'computed_session_id',
    F.concat_ws('_', 'user_id', 'session_seq')
)

# Aggregate to session-level metrics
df_session_agg = df_sessions.groupBy(
    'computed_session_id', 'user_id', 'device_type', 'country_code'
).agg(
    F.min('timestamp_utc').alias('session_start'),
    F.max('timestamp_utc').alias('session_end'),
    F.count('event_id').alias('event_count'),
    F.countDistinct('page_url').alias('unique_pages'),
    F.collect_set('event_type').alias('event_types'),
    F.max(F.when(F.col('event_type')=='purchase',1).otherwise(0)).alias('converted'),
    F.sum(F.when(F.col('event_type')=='purchase',
          F.col('properties')['revenue'].cast('double')).otherwise(0)
    ).alias('session_revenue')
).withColumn(
    'session_duration_mins',
    (F.col('session_end').cast('long') -
     F.col('session_start').cast('long')) / 60
)

df_session_agg.write.format('delta') \
    .mode('overwrite') \
    .option('replaceWhere', f"date(session_start) = '{processing_date}'") \
    .save(f'{LANDING}/Tables/silver_sessions')
```

6.3 Delta Lake OPTIMIZE & VACUUM

PySpark + SQL — Optimize Delta Table Performance

```
# Notebook: 04_optimize_delta_tables.ipynb
# Run after loading Gold layer

tables_to_optimize = [
    'bronze_clickstream',
    'silver_sessions',
    'gold_clickstream_sessions',
]

for table in tables_to_optimize:
    print(f'Optimizing {table}...')

    # OPTIMIZE compacts small files into 1GB target size
    # Z-ORDER clusters data for column-predicate speedup
    spark.sql(f'''
        OPTIMIZE lakehouse.{table}
```

```
        ZORDER BY (event_date, user_id)
    ''')

    # VACUUM removes files older than 7 days
    spark.sql(f'''
        VACUUM lakehouse.{table} RETAIN 168 HOURS
    ''')

    # Show table details
    spark.sql(f'DESCRIBE DETAIL lakehouse.{table}').show(truncate=False)

print('All tables optimized.')

# Enable Auto-Optimize for ongoing ingestion
spark.sql('''
    ALTER TABLE lakehouse.bronze_clickstream
    SET TBLPROPERTIES (
        'delta.autoOptimize.optimizeWrite' = 'true',
        'delta.autoOptimize.autoCompact'   = 'true'
    )
''')
```

Chapter 7: Scenario 5 — Real-Time Streaming ETL

SCENARIO 5 | IoT Sensor Real-Time Streaming with EventStream

Business Problem: An IoT manufacturing plant sends machine telemetry (temperature, pressure, vibration) from 2,000 sensors at 1-second intervals via Azure Event Hubs. The data must be processed in near-real-time to detect anomalies and feed live dashboards.

Source: Azure Event Hubs — 2,000 sensors, ~2 million events/hour

Processing: EventStream → KQL Database + Spark Structured Streaming

Output: Real-Time Dashboard + Lakehouse Delta table for historical analysis

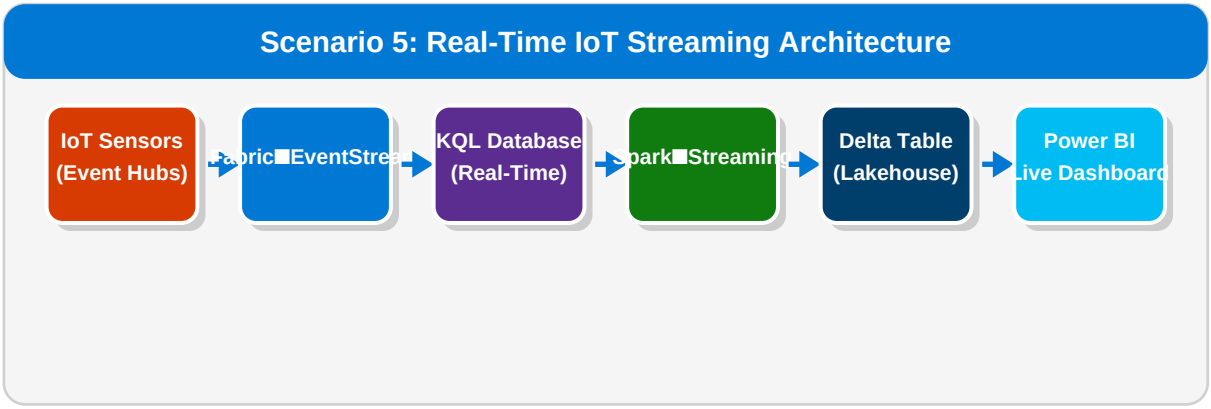


Figure 7.1 Dual-path streaming: KQL for real-time dashboards, Delta for historical queries.

7.1 Spark Structured Streaming — Ingest from Event Hubs

PySpark Structured Streaming — Read from Event Hubs

```
# Notebook: streaming_iot_ingest.ipynb (Always-On / attached to pipeline)
from pyspark.sql import functions as F
from pyspark.sql.types import *

# Event Hubs connection via Spark connector
EH_NAMESPACE   = 'iot-plant-eventhubs'
EH_NAME        = 'machine-telemetry'
EH_CONN_STRING = mssparkutils.credentials.getSecret('kv-name', 'eh-conn-string')

eh_conf = {
    'eventhubs.connectionString': sc._jvm.org.apache.spark.eventhubs
        .EventHubsUtils.encrypt(EH_CONN_STRING),
    'eventhubs.consumerGroup': '$Default',
    'eventhubs.startingPosition': '{"offset": "-1", "seqNo": -1,',
                                  '"enqueuedTime": null, "isInclusive": true}',
}
```

```
# Schema for telemetry payload
TELEMETRY_SCHEMA = StructType([
    StructField('sensor_id',    StringType(), False),
    StructField('machine_id',   StringType(), False),
    StructField('timestamp',    TimestampType(), False),
    StructField('temperature',  DoubleType(), True),
    StructField('pressure',     DoubleType(), True),
    StructField('vibration',    DoubleType(), True),
    StructField('rpm',          IntegerType(), True),
    StructField('status',       StringType(), True),
])

# Read stream
df_stream = (spark.readStream
    .format('eventhubs')
    .options(**eh_conf)
    .load()
    .select(F.from_json(
        F.col('body').cast('string'),
        TELEMETRY_SCHEMA
    ).alias('data'), 'enqueuedTime')
    .select('data.*', 'enqueuedTime')
)

# Add watermark for late data handling (allow 5 min lateness)
df_watermarked = df_stream \
    .withWatermark('timestamp', '5 minutes')

# Windowed aggregations — 1-minute tumbling windows
df_agg = (df_watermarked
    .groupBy(
        F.window('timestamp', '1 minute'),
        'machine_id', 'sensor_id'
    ).agg(
        F.avg('temperature').alias('avg_temp'),
        F.max('temperature').alias('max_temp'),
        F.avg('pressure').alias('avg_pressure'),
        F.avg('vibration').alias('avg_vibration'),
        F.avg('rpm').alias('avg_rpm'),
        F.count('*').alias('reading_count'),
        # Anomaly flag: temp > 95 or vibration > 8.0
        F.max(F.when(
            (F.col('temperature') > 95) | (F.col('vibration') > 8.0),
            1).otherwise(0)).alias('has_anomaly')
    )
    .withColumn('window_start', F.col('window.start'))
    .withColumn('window_end',   F.col('window.end'))
    .drop('window')
)

LAKEHOUSE = 'abfss://workspace@onelake.dfs.fabric.microsoft.com/lakehouse.Lakehouse'

# Write aggregated stream to Delta (every 30 seconds)
query = (df_agg.writeStream
    .format('delta')
    .outputMode('append')
    .option('checkpointLocation',
            f'{LAKEHOUSE}/Files/_checkpoints/iot_agg')
    .trigger(processingTime='30 seconds')
    .start(f'{LAKEHOUSE}/Tables/silver_iot_telemetry_agg')
```

```
)

query.awaitTermination()
```

7.2 KQL — Real-Time Anomaly Detection Query

KQL — Anomaly Detection on Live Sensor Data

```
// KQL Database query for Real-Time Dashboard
// Runs continuously via Real-Time Intelligence

MachineTelemetry
| where timestamp > ago(1h)
| extend
    is_high_temp     = temperature > 95.0,
    is_high_vibration = vibration  > 8.0,
    is_high_pressure  = pressure   > 150.0
| summarize
    avg_temp       = avg(temperature),
    max_temp       = max(temperature),
    anomaly_count  = countif(is_high_temp or is_high_vibration or is_high_pressure),
    total_readings = count()
    by machine_id, bin(timestamp, 1m)
| where anomaly_count > 0
| order by timestamp desc, anomaly_count desc

// Alert rule: trigger Data Activator when anomaly_count > 5
// in any 1-minute window for a single machine
```

Chapter 8: Scenario 6 — Full Medallion Architecture

SCENARIO 6 | Enterprise Medallion Architecture — Full End-to-End

Business Problem: A financial services firm needs a complete, governed data platform covering 15 data sources, 200+ tables, serving 50 Power BI reports and 3 ML models. They require data quality gates at each layer, full lineage, and SLA monitoring.

Scale: 15 sources, 200+ tables, ~1TB new data per day

Teams: Data Engineering, Analytics, ML, Compliance

Governance: Microsoft Purview integration, column-level lineage

8.1 Bronze / Silver / Gold Layer Design

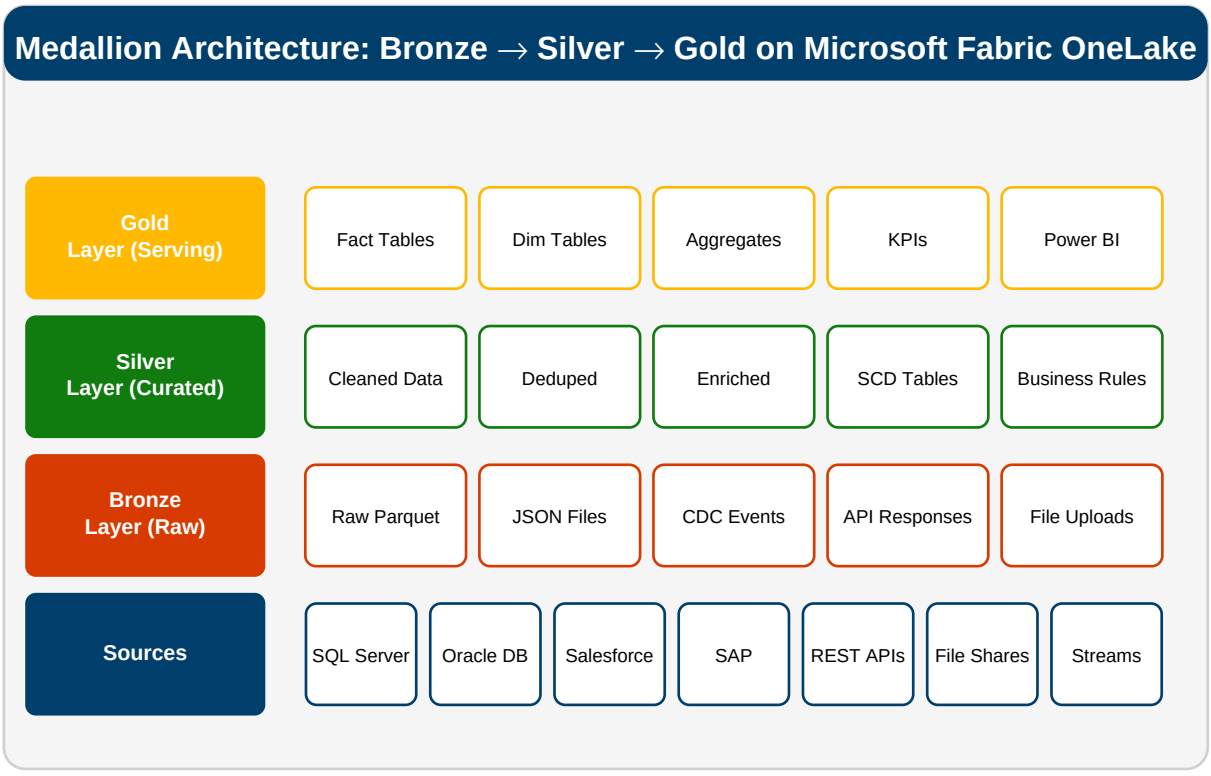


Figure 8.1 Three-tier Medallion architecture with source systems on OneLake.

8.2 Master Orchestration Pipeline

The master pipeline is triggered at 22:00 UTC and orchestrates all child pipelines in the correct dependency order. It uses **Execute Pipeline** activities for isolation and **If Condition** gates to halt execution if a critical

upstream layer fails.

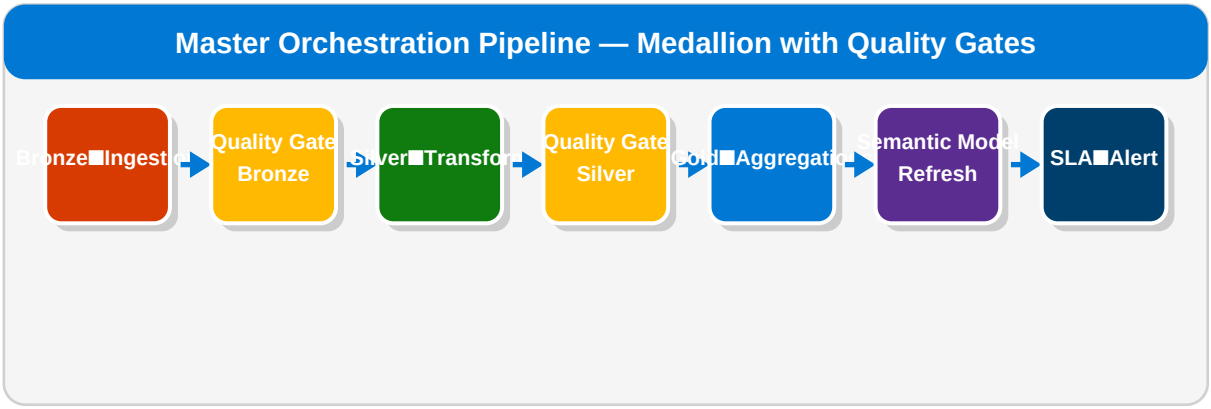


Figure 8.2 Master pipeline with quality gates between each Medallion layer.

8.3 Data Quality Checks with Great Expectations

PySpark — Great Expectations Data Quality Gates

```
# Notebook: data_quality_gate_silver.ipynb
import great_expectations as ge
from great_expectations.dataset import SparkDFDataset

def run_quality_gate(df, table_name, expectations):
    """Run Great Expectations checks and fail pipeline if threshold missed."""
    ge_df = SparkDFDataset(df)
    results = []
    for exp in expectations:
        result = getattr(ge_df, exp['type'])(**exp['kwargs'])
        results.append({
            'expectation': exp['type'],
            'success':     result['success'],
            'element_count': result.get('result', {}).get('element_count', 0),
            'unexpected_pct': result.get('result', {}).get('unexpected_percent', 0),
        })

    # Write quality report to Delta
    report_df = spark.createDataFrame(results)
    report_df.withColumn('table_name', F.lit(table_name)) \
             .withColumn('run_ts', F.current_timestamp()) \
             .write.format('delta').mode('append') \
             .save(f'{LAKEHOUSE}/Tables/dq_results')

    # Fail if any critical check failed
    critical_failures = [r for r in results
        if not r['success'] and r['unexpected_pct'] > 5.0]
    if critical_failures:
        raise Exception(
            f'Quality gate FAILED for {table_name}: {critical_failures}')
    return results

# Define expectations for silver_orders
ORDERS_EXPECTATIONS = [
    {'type': 'expect_column_values_to_not_be_null',
     'kwargs': {'column': 'OrderID'}},
    {'type': 'expect_column_values_to_not_be_null',
     'kwargs': {'column': 'CustomerID'}},
    {'type': 'expect_column_values_to_be_between',
     'kwargs': {'column': 'TotalAmount', 'min_value': 0, 'max_value': 1000000}},
    {'type': 'expect_column_values_to_be_in_set',
```

```
     'kwargs': {'column': 'Status',
                'value_set': ['Pending','Confirmed','Shipped','Cancelled']}},
    {'type': 'expect_table_row_count_to_be_between',
     'kwargs': {'min_value': 1000, 'max_value': 10000000}},
]

df_orders = spark.read.format('delta') \
    .load(f'{LAKEHOUSE}/Tables/silver_orders')

run_quality_gate(df_orders, 'silver_orders', ORDERS_EXPECTATIONS)
print('Quality gate passed for silver_orders!')
```

8.4 SLA Monitoring & Alerting via Logic Apps

Pipeline — Web Activity: SLA Alert on Completion

```
// Web Activity: SendSLAReport
// Runs on pipeline Completion (success or failure)
{
  'name': 'SendSLAReport',
  'type': 'WebActivity',
  'dependsOn': [
    { 'activity': 'GoldAggregations', 'dependencyConditions': ['Completed'] }
  ],
  'settings': {
    'url': 'https://prod.logic.azure.com/workflows/sla-monitor/triggers/...',
    'method': 'POST',
    'body': {
      'value': '{
        "pipeline_run_id": "@{pipeline().RunId}",
        "pipeline_name":   "@{pipeline().Pipeline}",
        "trigger_time":    "@{pipeline().TriggerTime}",
        "completion_time": "@{utcNow()}",
        "bronze_status":   "@{activity(''BronzeIngestion'').Status}",
        "silver_status":   "@{activity(''SilverTransform'').Status}",
        "gold_status":     "@{activity(''GoldAggregations'').Status}",
        "sla_met":         "@{less(int(activity(''GoldAggregations'').output.durationInQueue),
 14400)}"
      }',
      'type': 'Expression'
    }
  }
}
```

Chapter 9: Advanced Pipeline Patterns

9.1 Dynamic Pipelines — Full Metadata-Driven Framework

In a metadata-driven pipeline, the pipeline logic is completely generic — no hard-coded table names, schemas, or transformations. All configuration lives in a control database. Adding a new source table requires only inserting a row, not modifying any pipeline code.

SQL — Full ETL Metadata Framework Schema

```
-- Control database tables
CREATE TABLE meta.data_sources (
    source_id        INT  IDENTITY PRIMARY KEY,
    source_name      VARCHAR(100) NOT NULL,
    connection_type  VARCHAR(50),   -- SQL, REST, BLOB, SFTP
    linked_service   VARCHAR(100),
    is_active        BIT DEFAULT 1
);

CREATE TABLE meta.table_config (
    config_id          INT  IDENTITY PRIMARY KEY,
    source_id          INT  REFERENCES meta.data_sources(source_id),
    source_object      VARCHAR(200),  -- schema.table or endpoint path
    target_lakehouse   VARCHAR(100),
    target_table       VARCHAR(200),
    load_type          VARCHAR(20) DEFAULT 'INCREMENTAL',
    watermark_column   VARCHAR(100),
    primary_keys       VARCHAR(500),  -- comma-separated
    transform_notebook VARCHAR(200),  -- notebook to call post-copy
    dq_expectation_set VARCHAR(100),
    priority           INT DEFAULT 50,
    depends_on         INT REFERENCES meta.table_config(config_id),
    is_active          BIT DEFAULT 1
);

CREATE TABLE meta.pipeline_runs (
    run_id          UNIQUEIDENTIFIER PRIMARY KEY,
    config_id       INT,
    start_time      DATETIME2,
    end_time        DATETIME2,
    rows_read       BIGINT,
    rows_written    BIGINT,
    status          VARCHAR(20),  -- RUNNING, SUCCESS, FAILED, SKIPPED
    error_message   NVARCHAR(MAX)
);
```

9.2 Parallel Fan-Out / Fan-In Pattern

When multiple independent transformation jobs can run concurrently, use the **Fan-Out** pattern: start all jobs in parallel, then use a **Fan-In** join point (a script or notebook) that waits for all to complete before proceeding.

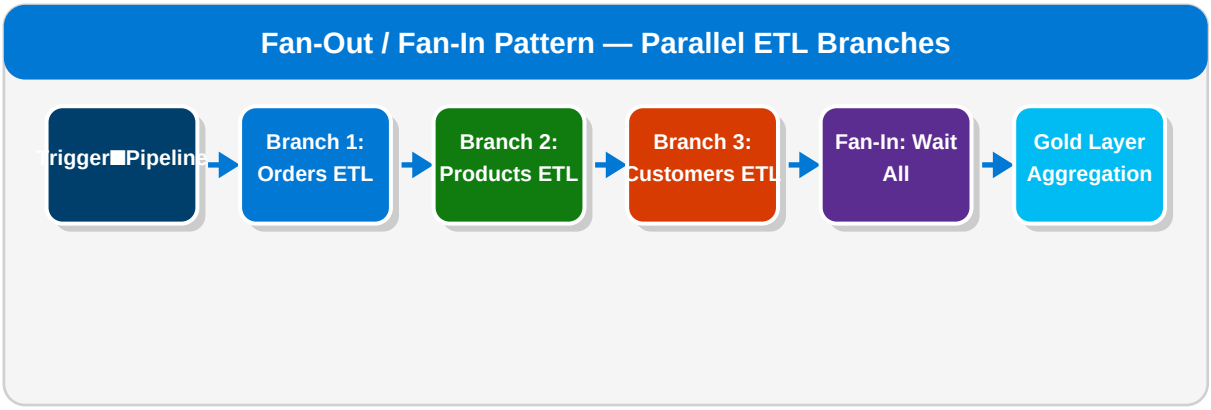


Figure 9.1 Three parallel ETL branches merge at a Fan-In aggregation step.

Pipeline JSON — Fan-Out Execute Pipeline Activities

```
// Three Execute Pipeline activities with NO dependencies on each other
// = they run in parallel when pipeline starts

{ 'name': 'RunOrdersETL',
  'type': 'ExecutePipeline',
  'settings': { 'pipeline': { 'referenceName': 'pl_orders_etl' }, 'waitOnCompletion': true }
},

{ 'name': 'RunProductsETL',
  'type': 'ExecutePipeline',
  'settings': { 'pipeline': { 'referenceName': 'pl_products_etl' }, 'waitOnCompletion': true }
},

{ 'name': 'RunCustomersETL',
  'type': 'ExecutePipeline',
  'settings': { 'pipeline': { 'referenceName': 'pl_customers_etl' }, 'waitOnCompletion': true
}
},

// Fan-In: GoldAggregation depends on ALL three completing
{ 'name': 'GoldAggregation',
  'type': 'TriggerRun',
  'dependsOn': [
    { 'activity': 'RunOrdersETL',    'dependencyConditions': ['Succeeded'] },
    { 'activity': 'RunProductsETL',  'dependencyConditions': ['Succeeded'] },
    { 'activity': 'RunCustomersETL', 'dependencyConditions': ['Succeeded'] }
  ]
}
```

9.3 CI/CD for Fabric Pipelines — Git Integration

Microsoft Fabric supports Git integration (Azure DevOps / GitHub) at the workspace level. Every pipeline, notebook, and dataflow definition is serialized to JSON/IPYNB and committed to the repo. CI/CD pipelines can validate, test, and deploy across environments using the Fabric REST API.

Azure DevOps YAML — Fabric Pipeline CI/CD

```
# azure-pipelines.yml
trigger:
  branches:
    include: [ main, release/* ]

variables:
  FABRIC_TENANT_ID:  $(tenantId)
```

```
  FABRIC_CLIENT_ID:  $(clientId)
  DEV_WORKSPACE_ID:  $(devWorkspaceId)
  PROD_WORKSPACE_ID: $(prodWorkspaceId)

stages:
  - stage: Validate
    jobs:
      - job: LintAndTest
        steps:
          - script: |
              pip install pytest nbformat pyyaml
              python -m pytest tests/ -v --tb=short
            displayName: 'Run notebook unit tests'

  - stage: DeployToDev
    dependsOn: Validate
    jobs:
      - deployment: DeployDev
        environment: development
        strategy:
          runOnce:
            deploy:
              steps:
                - script: |
                    # Get AAD token for Fabric REST API
                    TOKEN=$(curl -s -X POST \
                      https://login.microsoftonline.com/$(tenantId)/oauth2/v2.0/token \
                      -d 'client_id=$(clientId)&client_secret=$(clientSecret)' \
                      -d 'grant_type=client_credentials' \
                      -d 'scope=https://api.fabric.microsoft.com/.default' \
                      | jq -r .access_token)
                    # Trigger workspace Git sync
                    curl -X POST \
                      'https://api.fabric.microsoft.com/v1/workspaces/$(devWorkspaceId)/git/up
dateFromGit' \
                      -H "Authorization: Bearer $TOKEN" \
                      -H 'Content-Type: application/json' \
                      -d '{"workspaceHead": {"sourceControlProperties": {"branch": "main"}}}'
                  displayName: 'Deploy to DEV workspace via Fabric API'

  - stage: DeployToProd
    dependsOn: DeployToDev
    condition: and(succeeded(), eq(variables['Build.SourceBranch'], 'refs/heads/main'))
    jobs:
      - deployment: DeployProd
        environment: production
        strategy:
          runOnce:
            deploy:
              steps:
                - script: echo 'Deploy to PROD workspace'
```

Chapter 10: Monitoring, Testing & Best Practices

10.1 Pipeline Monitoring Dashboard

Every pipeline run in Fabric is logged to the monitoring hub. You can query run history, activity duration, and errors via the Fabric REST API or by connecting directly to the pipeline run history endpoint in a monitoring notebook.

PySpark — Build Custom Pipeline Monitoring Dashboard

```
# Notebook: monitoring_dashboard.ipynb
# Query Fabric pipeline run history via REST API
import requests, json
from pyspark.sql import functions as F

TOKEN = mssparkutils.credentials.getToken('https://api.fabric.microsoft.com')
WS_ID = mssparkutils.env.getWorkspaceId()

def get_pipeline_runs(pipeline_name, days_back=7):
    url = (f'https://api.fabric.microsoft.com/v1/workspaces/{WS_ID}'
           f'/items?type=DataPipeline')
    headers = {'Authorization': f'Bearer {TOKEN}'}
    response = requests.get(url, headers=headers)
    pipelines = response.json().get('value', [])

    pipeline_id = next(
        (p['id'] for p in pipelines if p['displayName'] == pipeline_name),
        None
    )
    if not pipeline_id:
        return []

    runs_url = (f'https://api.fabric.microsoft.com/v1/workspaces/{WS_ID}'
                f'/items/{pipeline_id}/jobs/instances')
    runs_resp = requests.get(runs_url, headers=headers)
    return runs_resp.json().get('value', [])

# Build monitoring dataframe
runs = get_pipeline_runs('master_etl_pipeline')
df_runs = spark.createDataFrame(runs)

# Compute SLA compliance
df_sla = df_runs.withColumn(
    'duration_mins',
    (F.unix_timestamp('endTimeUtc') - F.unix_timestamp('startTimeUtc')) / 60
).withColumn(
    'sla_met',
    F.col('duration_mins') <= 120  # 2-hour SLA
).withColumn(
    'run_date', F.to_date('startTimeUtc')
)

# Daily SLA summary
df_summary = df_sla.groupBy('run_date').agg(
    F.count('*').alias('total_runs'),
    F.sum(F.col('sla_met').cast('int')).alias('sla_met_runs'),
```

```
    F.avg('duration_mins').alias('avg_duration_mins'),
    F.max('duration_mins').alias('max_duration_mins'),
    F.sum(F.when(F.col('status')=='Failed',1).otherwise(0)).alias('failed_runs')
)

df_summary.write.format('delta').mode('overwrite') \
    .save(f'{LAKEHOUSE}/Tables/monitoring_pipeline_sla')

display(df_summary.orderBy('run_date', ascending=False))
```

10.2 Performance Tuning Checklist

Copy Activity	Use parallel copies (degree of copy parallelism = 32+), enable staging for large loads, use binary copy for file-to-file scenarios, partition source queries by date range.
Spark Notebooks	Explicitly set schema (no inferSchema), partition Delta writes, use ZORDER on high-cardinality filter columns, cache intermediate DataFrames used multiple times.
Dataflows Gen2	Enable Staging mode for tables > 100K rows, use Query Folding where possible, disable unnecessary preview refreshes, split complex dataflows into smaller ones.
Delta Tables	Run OPTIMIZE weekly, use ZORDER on filter columns, set delta.autoOptimize.optimizeWrite=true for streaming, partition on date columns.
Pipeline Design	Use ForEach parallel batches (batch count 4-8), avoid synchronous Execute Pipeline for independent work, use Set Variable sparingly in tight loops.

10.3 Security Best Practices

- Store all secrets in **Azure Key Vault** — never hard-code credentials in notebooks or pipelines.
- Use **Managed Identity** for all Linked Services wherever the target service supports it.
- Apply **workspace-level RBAC**: Contributor for engineers, Viewer for analysts.
- Enable **Microsoft Purview Data Map** for automated lineage scanning across all Fabric items.
- Use **column-level security** in the Warehouse for PII data (SSN, email, DOB).
- Enable **Row-Level Security (RLS)** on Power BI Semantic Models for business-unit data isolation.
- Use **Private Endpoints** for all Linked Services connecting to Azure resources.
- Regularly rotate service principal secrets and store rotation dates in Key Vault tags.
- Enable **Diagnostic Logs** and send to Log Analytics for audit trail retention (90+ days).

10.4 Quick Reference: Pipeline Expression Functions

Expression	Description
`@utcNow()`	Current UTC datetime as string
`@formatDateTime(utcNow(),'yyyy-MM-dd')`	Format datetime to date string
`@pipeline().RunId`	Current pipeline run GUID
`@pipeline().TriggerTime`	Trigger timestamp
`@activity('Name').output.X`	Access activity output property
`@activity('Name').Status`	Activity status: Succeeded/Failed/Skipped
`@concat('a','b','c')`	Concatenate strings
`@if(equals(x,y), 'yes', 'no')`	Inline conditional expression
`@length(array)`	Count elements in array
`@item()`	Current ForEach iterator item
`@json(string)`	Parse JSON string to object
`@string(value)`	Convert value to string
`@int(value)`	Convert value to integer
`@add(a,b)`	Add two numbers
`@split(str,',')`	Split string to array
`@contains(collection,value)`	Check if collection contains value

Summary: Microsoft Fabric ETL at a Glance

OneLake • Delta-Parquet • Open Format • Zero Duplication

Data Factory Pipelines • Dataflows Gen2 • Spark Notebooks

Bronze → Silver → Gold • Real-Time EventStream • KQL Analytics

Git Integration • CI/CD • Purview Governance • Power BI Direct Lake